

Understanding ACT: Inference

Explaining the Style Variable Generation

1 The Simplified Forward Pass

Once the model is successfully trained, the complex Dual-Learning setup is dismantled for inference. Because the human expert is no longer demonstrating the task, we do not have future actions to look at. Therefore, the **Style Encoder** is mathematically disabled and thrown away.

To fill the missing z parameter required by the Policy Transformer, we rely on the power of the KL Divergence prior established during training. We simply hardcode the style variable to the origin: $z = \vec{0}$. This guarantees that the Policy will execute the most robust, neuro-normative, "average" version of the strategy it learned.

The inference forward pass thus becomes incredibly simple:

1. Read current cameras and proprioception.
2. Pass them, along with $z = \vec{0}$, into the Policy Encoder to build context.
3. The Policy Decoder reads the context and outputs a full predicted **Action Chunk** (e.g., $k = 100$ actions).

2 The Overlapping Chunk Problem

A naive implementation would execute all 100 predicted actions sequentially, wait for them to finish, and then take another picture to predict the next 100. However, this causes robotic movement to be incredibly jerky and completely blind to sudden changes in the environment while executing the chunk.

To achieve smooth, highly-reactive motion, the system runs inference continuously at a very high frequency (e.g., querying the policy every few milliseconds).

Because the model predicts 100 future steps, but we query it again almost immediately, **the predictions overlap**. For example, at physical timezone $t = 10$:

- The policy from timestep $t = 0$ predicted what action 10 should be.
- The policy from timestep $t = 2$ predicted what action 10 should be.
- The policy from timestep $t = 9$ predicted what action 10 should be.

We now have a queue of multiple different predictions competing for the exact same physical motor execution!

3 Temporal Ensembling

To solve the overlapping problem, ACT uses a technique called **Temporal Ensembling**. Instead of just playing the newest action and discarding the old ones, it mathematically blends them together using an **Exponential Weighting Scheme**.

For any given timestep t , the final executed action a_t is a weighted average of all the overlapping predictions from previous chunks.

$$a_t = \frac{\sum w_i A_t[i]}{\sum w_i}$$

Where $A_t[i]$ represents the competing predictions for time t , and w_i is an exponential decay weight:

$$w_i = \exp(-m \cdot i)$$

- **High Weights (w_i):** The oldest predictions (made several steps ago) are given the highest weight, as they form the stable "core" of the trajectory over time.
- **Low Weights (w_i):** The newest, most recent predictions are given less weight. They act as "corrective nudges" to adjust mathematically for sudden changes without violently pulling the robot off its stable path.

This asynchronous overlap of k buffered actions smoothed by exponential averaging is what allows ACT policies to be both long-horizon (seeing 100 steps into the future) and incredibly smooth and reactive in the present.